ROBOPharma GUI

Technical Manual

MAY 2026

SOFTWARE VERSION 1.0

WRITTEN BY ROBERTA GRIFFITHS

CONTENTS

01

SYSTEM ARCHITECTURE

The ROBOPharma GUI consists of three main nodes, a simulation, and a database. **CORE COMPONENTS** The system is divided into five primary subsystems that operate concurrently:

1. **Simulation Environment (CoppeliaSim):** Acts as a simulated version of the main rig. It handles the kinematics of the robotic arms (uArm), the conveyor mechanism, and the spinner assembly. It publishes state changes and simulation camera feeds while listening for sorting commands.
2. **Logic Node:** The central coordinator of the system. It controls the simulation, determines the active stage (Load, Conveyor, Spin, Detect, Unload), and broadcasts these state transitions to the rest of the network.
3. **Data Logger:** Listens to the data streams generated by the Logic node and rig, writing stage timings and detection metrics to a local SQLite database (results.db) and saving localised analysis images.
4. **Database:** SQLite database where data is saved, and can be later accessed by the GUI.
5. **GUI Dashboard Node:** A NiceGUI web server acting as the bridge between the ROS 2 backend and the operator. It subscribes to ROS topics to show a live dashboard and queries the SQLite database directly to generate historical performance charts and PDF reports.

02

LOGIC NODE

The Logic Node is the central state machine and coordinator for system. It subscribes to topics and creates new message structures to control the GUI and simulation.

SUBSCRIPTIONS

Topic Name	Message Type
/loading_status	std_msgs/String (JSON)
/unloading_status	std_msgs/String (JSON)

/vision_system/processing_time	std_msgs/Float32
/bag_contamination_status	std_msgs/String (JSON)
/current_position	std_msgs/Float32

PUBLISHERS

Topic Name	Message Type	Purpose
/trigger_platform_move	std_msgs/Int32	Commands the conveyor/platform (1 = Forward, 2 = Backwards, 0 = Inactive).
/spinner	std_msgs/Int32	Triggers the spinning mechanism (1 = Start).
/system_state	custom_interfaces/ SystemStatus	Broadcasts the currently active state as boolean flags.
/state_timings	custom_interfaces/ StateTimings	Publishes the accumulated duration (in seconds) for each state and the total process time.
/detection_status	custom_interfaces/ DetectionStatus	Details the detection result for each IV bag.
/detection_setup	custom_interfaces/ DetectionSetup	Details the detection setup information, including the particle and confidence threshold as well as model used.
/bag_number	custom_interfaces/ BagNumber	Tracks the current bag index out of the total for the cycle.
/test_result	std_msgs/Int32	Publishes the final binary result (1 = Contaminated, 0 = Clean) during unloading for the simulation.
/reset_simulation	std_msgs/Int32	Triggers a full system reset (1) once a cycle is complete.

STATE LOGIC

The node maintains a state machine driven by incoming callbacks. The states are tracked in self.active_state_name and broadcasted via the /system_state topic.

State	Triggering Topic	Data Condition
Load	/loading_status	phase == "positioning"
Conveyor	/loading_status	phase == "motor_advancing"
Spin	/current_position	data > 0.0

Detect	/vision_system/ processing_time	data > 0.0 (And current state is "spin")
Unload	/unloading_status	status == "in_progress"
Idle		When above states are complete and before the next is active

1. **load**: Initiated when the loader is positioning or loading a bag.
2. **conveyor**: Initiated when the loading motor is advancing.
3. **spin**: Initiated spinner is actively rotating to agitate particles
4. **detect**: Active when vision system is processing
5. **unload**: Initiated when bags are unloaded from the spinner
6. **IDLE (None)**: The system returns to a blank/inactive state upon cycle completion or when waiting for the next state.

TIMING CALCULATIONS

Every time the system transitions between states via update_system_state(), the node calculates the duration of the *ending* state and adds it to the cumulative time in StateTimings. Total cycle time is tracked from the moment the first state begins until the system is reset.

03

DATA LOGGER NODE

The Data Logger Node controls which information is saved into the database. It subscribes to various data streams (including detection metrics, state timings, and camera feeds) to store detection images locally and log cycle results into the SQLite database.

SUBSCRIPTIONS

Topic Name	Message Type	Purpose
/detection_status	custom_interfaces/ DetectionStatus	Receives detection metrics, statuses, and contamination levels for each IV bag.
/state_timings	custom_interfaces/ StateTimings	Receives the accumulated duration for each state and the total process time.

/vision_system/ processed_image/ compressed	sensor_msgs/ CompressedImage	Receives and saves the latest compressed image as a base64 data URI.
/detection_setup	custom_interfaces/ DetectionSetup	Saves the current detection conficuration including particle/confidence thresholds and the active model.
/current_position_step	std_msgs/ String (JSON)	Receives analysis state and description (e.g., "IV Bag 1 - Position 2") to trigger image saving.

PUBLISHERS

Topic Name	Message Type	Purpose
/system_errors	std_msgs/String (JSON)	Broadcasts formatted JSON payloads containing system errors, unhandled exceptions, and DB failures.

DATA STORAGE LOGIC

The node maintains a local directory structure (logs/images) and an SQLite database (logs/results.db) driven by incoming callbacks.

Data Type	Triggering Topic	Storage Condition & Action
Analysis Images	/current_position_step	Triggered when state == "analyzing". Decodes the saved base64 image and saves it as a .jpg with cycle, bag, and position identifiers.
Detection Metrics	/detection_status	Inserts data into the detections table (e.g., particles, bubbles, confidence, thresholds). Links previously saved image paths to the database row.
Setup Config	/detection_setup	Saves the configuration in memory (self.latest_setup) to append to subsequent detections database writes.
Errors	Internally Triggered through Exception blocks	Caught exceptions format a compact JSON string detailing the error code, severity, and traceback, publishing it to /system_errors.

TIMING & CYCLE TRACKING CALCULATIONS

Every time the system receives timing data via the timing_callback, the node validates the durations.

- **Cycle Persistence:** A cycle's timings are only written to the cycles table once **all** stage timers (load, conveyor, spin, detect, unload) are strictly greater than 0.0 and haven't already been logged.
- **Cycle Incrementing:** The node tracks the last_processed_bag. If it receives data for Bag 1 after having processed a higher bag number (e.g., Bag > 1), it automatically increments the global current_cycle tracker and clears the cached image paths to prepare for the next batch. Upon initialization, it resumes the cycle count by querying the highest existing cycle in the database.

04

DATABASE

The system uses a local SQLite database (saved as results.db in the logs directory) to save data across runs. The schema consists of two primary tables: cycles for tracking system performance and stage timings, and detections for logging the detailed analysis results and configuration parameters of individual IV bags.

CYCLES TABLE

The cycles table records the duration of each operational stage to track system efficiency and total processing time. A row is inserted only upon the completion of a full cycle.

Column Name	Data Type	Description
id	INTEGER	Primary key and unique identifier for the cycle record (Auto-incremented).
timestamp	TEXT	Date and time the cycle timings were logged.
load_time	REAL	Duration (in seconds) of the bag loading phase.
conveyor_time	REAL	Duration (in seconds) of the conveyor motor advancing phase.
spin_time	REAL	Duration (in seconds) of the spinning/agitation phase.
detect_time	REAL	Duration (in seconds) of the vision system processing phase.
unload_time	REAL	Duration (in seconds) of the unloading phase.
process_time	REAL	Total cumulative time for the entire system cycle.

DETECTIONS TABLE

The detections table stores the detection results for every processed IV bag. It includes the analysis metrics, file paths to the localised images, and a snapshot of the detection setup configurations used at the time of processing.

Column Name	Data Type	Description
id	INTEGER	Primary key and unique identifier for the detection record (Auto-incremented).
cycle	INTEGER	The global cycle number this bag belongs to.
bag_number	INTEGER	The sequential index of the bag within its specific cycle.
timestamp	TEXT	Date and time the detection was recorded.
status	TEXT	The overall operational or pass/fail status of the bag.
contamination_level	TEXT	The categorized level of identified contamination.
recommendation	TEXT	Actionable recommendation based on the detection (e.g., Accept, Reject).
particles	INTEGER	Total count of identified particulate matter inside the bag.
bubbles	INTEGER	Total count of identified bubbles.
confidence	REAL	Average AI confidence score for the detections across both visual positions.
image_path_pos1	TEXT	Local file path to the saved .jpg analysis image for position 1.
image_path_pos2	TEXT	Local file path to the saved .jpg analysis image for position 2.
detection_model	TEXT	The name or version of the machine learning model used for this bag.
setup_particle_threshold	INTEGER	The active particle count limit required to trigger a contamination status.
setup_confidence_threshold	REAL	The active minimum confidence threshold required for a valid detection.

05

GUI NODE

The GUI Node acts as the bridge between the ROS 2 system, local database storage, and the web-based NiceGUI interface. Rather than updating the UI directly upon receiving ROS messages, the node stores the latest system state internally. Periodic UI timers then poll this shared state to refresh page widgets, ensuring a responsive front-end that does not block ROS 2 execution.

SUBSCRIPTIONS

Topic Name	Message Type	Purpose
/system_state	custom_interfaces/ SystemStatus	Tracks the currently active mechanical stage using boolean flags.
/detection_status	custom_interfaces/ DetectionStatus	Receives live pass/fail results, contamination levels, and metrics per bag to update global counters.
/detection_setup	custom_interfaces/ DetectionSetup	Saves the active detection setup (model name, particle/confidence thresholds).
/vision_system/ processed_image/ compressed	sensor_msgs/ CompressedImage	Saves the latest detection camera frame, converting it to a base64 URI for the UI.
/simulation_view	sensor_msgs/ CompressedImage	Saves the latest simulation render, converting it to a base64 URI for the UI.
/state_timings	custom_interfaces/ StateTimings	Receives live cycle durations to update the home dashboard progress bars.
/bag_number	custom_interfaces/ BagNumber	Tracks the current and total bag counts.
/vision_system/ current_model	std_msgs/String	Listens for the currently active vision model name to display in settings.
/system_errors	std_msgs/String (JSON)	Listens for JSON-formatted system errors, triggering the top-right UI alert indicator and logging them locally.

CLIENTS

Name	Type	Purpose
/switch_model	Service Client (std_srvs/Trigger)	Requests a parameter/model cycle from the backend. Guarded by an offline check to prevent UI hangs.

DASHBOARD PAGES & UI LOGIC

The NiceGUI application runs on port 8080 and is divided into six primary pages, all sharing a sidebar and an active status/error indicator.

Home (/)

- The main live operations dashboard.
- It displays performance metrics (Bags Processed, Pass Rate, Process Time, Failed)
- Visualises real-time stage progress via adaptive loading bars
- Highlights the currently active operational stage
- Provides a scaled view of the live simulation
- Polled at 10Hz (0.1s).

Detection (/detection)

- Focuses on bag inspection output
- It displays a live pass/fail pie chart, exact bubble/particle counts, analysis time, and confidence metrics
- Shows the active thresholds
- Shows live camera feed used during detection
- Polled at 10Hz.

Metrics (/metrics)

- A historical analysis dashboard
- Queries the local SQLite results.db directly (bypassing ROS topics) to populate charts
- It features a Timeframe toggle (1 Day, 1 Week, 1 Month) that filters SQL queries
- Shows Pass/Fail trends, Particles/Bubbles over time, and cycle stage timings
- Polled at 0.5Hz (2.0s).

Simulation (/simulation)

- A dedicated, full-screen view of the /simulation_view topic stream for observation of the simulation.

Downloads (/downloads)

- A report browser querying the detections table in results.db

- Users can view historical records, select specific runs, and generate/download formatted .pdf reports
- The ReportLab generation compiles the database row data and appends the localized image snapshots (image_path_pos1 & image_path_pos2) into the final document.
- The table auto-refreshes every 5s.

Settings (/settings)

- Provides system controls, allowing the user to view the currently active detection model and trigger a Cycle Model command via the /switch_model service.
- Included safeguards notify the user gracefully if the backend service is offline.

06

SIMULATION

The simulation environment in CoppeliaSim is driven by three primary Lua scripts, a central system coordinator that manages ROS 2 communications and the global state machine, and dedicated robotic arm controllers that handle inverse kinematics and pick-and-place sequences. These scripts communicate with each other internally using CoppeliaSim's integer signals (e.g., uArm_Start_Command, uArm_Finished).

MAIN CONTROLLER

This script acts as the simulations master node. It subscribes to ROS 2 topics to mirror the physical rig's state, controls the visual indicators, spawns the IV bags, and orchestrates the linear actuators and spinner motors.

SUBSCRIPTIONS

Topic Name	Message Type	Purpose
/system_state	custom_interfaces/ SystemStatus	Receives the current active state to trigger visual color changes (active = blue, inactive = grey) on the simulation objects.
/trigger_platform_move	std_msgs/Int32	Receives platform movement commands to coordinate the loading phase.

/test_result	std_msgs/Int32	Receives the binary pass/fail result (1 or 0) to determine how the unload arm should sort the bag.
/reset_simulation	std_msgs/Int32	Listens for a reset flag (1) to interrupt the current cycle, stop motors, and clean up the environment.

PUBLISHERS

Topic Name	Message Type	Purpose
/system_errors	std_msgs/String	Broadcasts JSON-formatted simulation lifecycle events, such as simulator_started and simulator_stopped.
/simulation_view	sensor_msgs/ CompressedImage	Broadcasts the images from the camera in the simulation, to provide a live view

OPERATIONAL PHASES

The main thread runs a continuous loop governed by the incoming /system_state flags:

- **Initialisation & Spawning:** Waits for an active state, clears any existing IV bags, and clones 5 new prototype bags at predefined coordinates.
- **Loading:** Iterates through the spawned bags, signaling the uArm to execute its pick-and-place sequence. Once placed, the bags are parented to the spinner assembly and locked in place.
- **Spinning:** When systemStatus.spin is true, the script disables positional control on the spinner's revolute joint and sets a target velocity (10.0) to simulate rotation.
- **Unloading & Sorting:** Stops the spinner and waits for a /test_result. It then signals the unloading robot (uArm_unload), passing the sort type (pass/fail). After the robot finishes, the spinner rotates by 60° to move to the next bag.

ROBOTIC ARMS

This script is attached to the robotic arms and handles the physical manipulation of the IV bags. It translates high-level commands from the main controller into smooth, multi-joint trajectories.

PICK AND PLACE SEQUENCE

Triggered when the main script sets the uArm_Start_Command signal to 2:

1. **Hover:** Moves to a calculated position directly above the target IV bag.
2. **Approach & Grip:** Moves straight down, activates the gripper, and parents the bag to the arm, disabling the bag's physics.
3. **Retract:** Moves straight up to the hover position.
4. **Transfer:** Calculates the current position of the moving platform and executes a trajectory to reach the drop-off coordinates.
5. **Release:** Disables the gripper, re-parents the bag to the platform, and re-enables the bag's physics.
6. **Cleanup:** Sets the uArm_Finished signal to 1 to notify the main controller and returns to its home position.